

AWS Capstone - Candidate Question Paper

Activities A1, A2 and A3

July 2026

Table of Contents

AWS Capstone — Candidate Question Paper

Activities A1, A2 (scope) and A3

ACTIVITY A1 — AWS Knowledge Assessment

Duration: 60 minutes | **Questions:** 35 | **Marks:** 1 each, no negative marking **Mode:** Individual, proctored, closed-book. No AWS console, no documentation, no AI assistants. **Pass indicator:** 60% (21/35). Below 40% triggers a mandatory re-sit before the POC demo.

Every question is single-best-answer. Several options are plausible; choose the one that represents correct AWS practice for the scenario as described.

Section A — Identity, Access & Security (5 marks)

Q1. A containerised service running on Fargate needs read access to an object storage bucket. What is the correct way to grant it? - a) Attach an IAM role to the underlying EC2 instance - b) Bake IAM user access keys into the container image - c) Assign an IAM task role in the task definition - d) Pass the access key and secret as task-definition environment variables

Q2. A database password must be rotated automatically every 30 days without application changes. Which service is designed for this? - a) AWS Secrets Manager - b) SSM Parameter Store, standard String parameter - c) An S3 object encrypted with SSE-KMS - d) A Lambda environment variable

Q3. A third party must assume a role in your account to run deployments. What should the trust relationship rely on? - a) A shared long-lived access key - b) A security group rule restricting the source - c) An IP-address condition alone - d) An `sts:AssumeRole` trust policy with an `ExternalId` condition

Q4. Your GitHub Actions pipeline must deploy to AWS with no long-lived credentials stored in the CI system. What do you configure? - a) An IAM user whose access key is held in GitHub Secrets - b) OIDC identity-provider federation to an IAM role - c) A Cognito identity pool - d) An EC2 instance profile on the runner

Q5. Which statement about IAM policy evaluation is correct? - a) An explicit Allow overrides an explicit Deny - b) A permissions boundary grants permissions to a principal - c) An explicit Deny always wins - d) Resource-based policies are ignored when an identity-based policy exists

Section B — Networking (4 marks)

Q6. Tasks in a private subnet must pull container images and ship logs, and the security team forbids any internet egress path. What do you provision? - a) A NAT Gateway - b) An Internet Gateway with a default route - c) A Site-to-Site VPN - d) Interface endpoints for the registry API, registry Docker, and logs — plus a gateway endpoint for object storage

Q7. Which correctly distinguishes security groups from network ACLs? - a) Security groups are stateful and applied at the network interface; NACLs are stateless and applied at the subnet - b) Security groups are stateless; NACLs are stateful - c) Both are stateless and must have explicit return rules - d) NACLs support allow rules only

Q8. A single load balancer must route `/api/incidents` to the Java service and `/api/resources` to the .NET service. Which feature does this? - a) Route 53 latency-based routing - b) Listener rules with path-pattern conditions - c) Weighted target groups - d) It is only possible with a CDN in front

Q9. A private service must call the secrets store without its traffic leaving the AWS network. What do you add? - a) A public IP on the task - b) A Direct Connect link - c) An interface VPC endpoint (PrivateLink) for the secrets service - d) Nothing — a NAT Gateway is the only supported option

Section C — Compute, Containers & Deployment (5 marks)

Q10. Image thumbnails must be generated on upload. Volume is bursty and each job runs under a second. Which compute model fits best? - a) An EC2 Auto Scaling group - b) ECS on EC2 with a warm pool - c) EKS with a cluster autoscaler - d) Lambda

Q11. You need blue/green deployment for an ECS service with automatic rollback triggered by a CloudWatch alarm. What provides this natively? - a) CodeDeploy as the ECS deployment controller - b) `aws ecs update-service` rolling update - c) Elastic Beanstalk immutable deployments - d) Lambda aliases with weighted routing

Q12. Your .NET container needs 40 seconds to become ready. The load balancer marks it unhealthy and the orchestrator kills it, looping forever. Best fix? - a) Increase the load balancer idle timeout - b) Disable health checks - c) Set an ECS health-check grace period and tune the health-check interval and unhealthy threshold - d) Migrate the service to Lambda

Q13. A latency-sensitive Java Lambda function suffers unacceptable cold starts. Which addresses this most directly? - a) Increase the function timeout - b) Increase reserved concurrency - c) Move dependencies into a layer - d) Enable provisioned concurrency and/or SnapStart

Q14. Why should container image tags be immutable in your registry? - a) It reduces storage cost - b) It is a requirement for Fargate - c) It guarantees the artefact you tested is the artefact you deployed - d) It enables cross-region replication

Section D — Data (5 marks)

Q15. Two dispatchers simultaneously try to allocate the last available ambulance. Using a NoSQL key-value store, what prevents a double allocation? - a) Read the item with a scan, then write - b) A conditional write on a version or status attribute (optimistic concurrency) - c) An eventually consistent read followed by a write - d) Adding a caching layer in front

Q16. Incident data needs multi-table joins, ACID transactions, ad-hoc reporting SQL, and the load is unpredictable and spiky. Which datastore? - a) A NoSQL key-value store - b) Object storage queried with a serverless SQL engine - c) An in-memory cache - d) Aurora Serverless v2, PostgreSQL-compatible

Q17. RDS Multi-AZ primarily provides: - a) A synchronous standby in another Availability Zone for automatic failover - b) Read scaling across replicas - c) Cross-region disaster recovery - d) Reduced storage cost

Q18. Browser clients must upload photos to object storage without the bytes passing through your API. What do you use? - a) Base64-encode the file into the API request body - b) A presigned URL or presigned POST issued by your API - c) A shared network file system mount - d) A bucket policy allowing public writes

Q19. Raw telemetry items in a NoSQL table must be deleted automatically 30 days after creation, at minimal cost. What do you use? - a) A nightly Lambda that scans and deletes - b) Change streams on the table - c) A time-to-live attribute with TTL enabled - d) Scheduled backup and restore

Section E — Integration & Event-Driven Design (5 marks)

Q20. A burst of 500 reports per minute must not lose a single record even if the downstream consumer is completely stopped. Which pattern? - a) Synchronous HTTP call from producer to consumer - b) Clients writing directly to the relational database - c) Scaling the load balancer - d) Queue-based load levelling with a durable queue and a dead-letter queue

Q21. One incident event must reach four independent consumers, each interested in a different subset of events, with no coupling between them and no change to producer code when a fifth is added. Best fit? - a) An event bus with rules and content-based event patterns - b) A single queue that all four consumers read from - c) A stream with four shards - d) A workflow orchestrator

Q22. Telemetry must be processed **in order per vehicle**, with the ability to **replay the last 24 hours** after a bug fix. Which is most appropriate? - a) A standard queue - b) A pub/sub topic - c) A FIFO queue with a single message group - d) A data stream partitioned by vehicle ID

Q23. A message has failed processing five times. What should happen and why? - a) Delete it, to keep the queue healthy - b) Route it to a dead-letter queue for inspection and later replay - c) Return it to the queue indefinitely - d) Log the payload and drop the message

Q24. How do you make report submission idempotent across unreliable client retries? - a) Use HTTP PUT instead of POST - b) Enable caching at the API layer - c) Have the client supply an idempotency key that the server records and checks with a conditional write - d) Assume clients will not retry

Section F — IaC & CI/CD (4 marks)

Q25. How should `terraform plan` be used in a team pipeline? - a) Auto-apply on every commit to any branch - b) Run plan on pull requests, publish the output for review, and gate apply behind an approval - c) Run it only against production - d) Skip it; apply reports the same information

Q26. Terraform state for a team of six must be: - a) Committed to Git alongside the code - b) Kept on the platform lead's laptop and shared on request - c) In a remote backend with state locking and encryption at rest - d) Deleted and regenerated on each run

Q27. Which is a valid canary release approach for a serverless API? - a) Alias weighted routing or an API canary stage shifting a small traffic percentage, with alarm-based automatic rollback - b) Deploy to production and monitor the logs manually - c) A DNS cutover between two full environments - d) Restarting the function after deployment

Q28. Which tooling detects security misconfiguration — such as a security group open to `0.0.0.0/0` — *before* the infrastructure is created? - a) `terraform fmt` - b) CloudWatch Logs Insights - c) A container image vulnerability scan - d) Checkov, tfsec or cfn-nag run as a pipeline stage

Section G — Observability & Operations (4 marks)

Q29. You must follow a single request from the API entry point, through the Java service, across a queue, into the .NET consumer. What gives you this? - a) Grepping centralised logs by timestamp - b) VPC Flow Logs - c) Distributed tracing with trace context propagated across the async hop - d) The account API audit log

Q30. Which metric best signals that a queue consumer cannot keep up with the producer? - a) Number of messages sent - b) Approximate age of the oldest message - c) Load balancer CPU utilisation - d) Bytes downloaded

Q31. CloudTrail's primary purpose is: - a) Collecting application stdout and stderr - b) Rendering metric dashboards - c) Distributed request tracing - d) An audit trail of which principal called which API, when

Q32. You want a business metric — “incidents triaged per minute”, broken down by priority — available for dashboards and alarms. What is the right approach? - a) Manually parse log files at the end of the day - b) Emit a custom metric with dimensions, via embedded metric format or a metrics API call - c) Count rows in the database on a schedule - d) Print the count to standard output

Section H — Cost & Well-Architected (3 marks)

Q33. In a short-lived development environment, which is typically the largest avoidable cost? - a) Object storage for a few gigabytes of photos - b) Applying cost allocation tags - c) NAT Gateways, idle managed databases and load balancers left running 24×7 - d) Using Spot capacity for containers

Q34. You must report exactly what this POC has spent, separate from everything else in the account. What enables that? - a) Cost allocation tags plus a filtered Cost Explorer view and a tag-scoped budget - b) A billing metric alarm on total account spend - c) Trusted Advisor's checks - d) Multiplying instance count by list price

Q35. Which of these is **not** an AWS Well-Architected Framework pillar? - a) Operational Excellence - b) Sustainability - c) Marketability - d) Reliability

— End of Activity A1 —

ACTIVITY A2 — Technical Panel Interview (scope only)

Duration: 45 minutes | **Panel:** 2 evaluators | **Weight:** 15%

Questions are **not published in advance**. You will be assessed on your ability to reason aloud, not to recite. The panel draws from:

Tier	Focus	Approx. time
Tier 1	Foundations — request lifecycle on AWS, identity model, compute selection, configuration and secrets, zero-downtime deployment, container troubleshooting	12 min
Tier 2	Design & diagnosis — scaling for a traffic spike, diagnosing a latency regression, guaranteeing no lost data, concurrency correctness, network isolation, cost investigation, distributed transactions	20 min
Tier 3	Your stack — deep questions specific to Java / .NET / Python / QA on AWS	13 min

How to prepare: be ready to whiteboard, to say “I don’t know, here’s how I’d find out,” and to defend a trade-off. The panel deliberately pushes back on correct answers to see whether you hold your position with reasoning or fold. Both folding and stubbornness score badly; reasoned defence scores well.

ACTIVITY A3 — Hands-On Skill Check

Duration: 120 minutes | **Mode:** Individual, AWS console + documentation permitted, **no AI assistants**, no collaboration | **Weight:** 15%

You are given a starter repository and a sandbox AWS account. Three independent parts — attempt them in any order. **Commit and push after each part**; uncommitted work is not assessed.

Part A — Fix the broken infrastructure code (40 min, 40 marks)

infra/ contains Terraform that provisions a VPC, a container service and a database. It applies successfully but contains **seven deliberate defects**. For each one: fix it, and add a one-line comment # FIX: <what was wrong and why it mattered>.

Defect categories in scope (the count per category is not given): - Over-privileged IAM - A resource exposed to the internet that should not be - A credential committed in plaintext - Missing encryption - No remote state or state locking - Missing required tags - An availability/durability setting left at an unsafe default

Marks: 5 for each defect correctly identified and fixed, up to 35. **5 further marks** for your `FINDINGS.md` ranking the defects by real-world severity with one line of justification each.

Finding a defect but “fixing” it in a way that breaks the deployment scores 2, not 5.

Part B — Containerise and deploy (50 min, 40 marks)

`app/` contains a small HTTP service **in your own stream’s language** (Java / .NET / Python) exposing `GET /health` and `GET /echo?msg=`. It currently runs only on a laptop.

Deliver:

1. A **Dockerfile** — multi-stage, non-root user, no build tooling in the final image, and a `HEALTHCHECK` (8 marks)
2. Image built and pushed to a private registry with an **immutable, traceable tag** — not `latest` (6 marks)
3. Service **running on Fargate in a private subnet**, behind a load balancer, healthy, reachable through the load balancer DNS name (12 marks)
4. The value returned by `GET /config` sourced from the **parameter/secrets store at runtime** — not from the image, not from a plaintext environment variable (8 marks)
5. Container logs visible in centralised logging, and one screenshot proving it (6 marks)

Evidence: the load balancer URL, and screenshots of a healthy target group and of your logs.

Part C — Make an async consumer safe (30 min, 20 marks)

`consumer/` is a queue consumer that crashes on any message where `payload.amount < 0`. Currently such a message blocks the queue forever.

Deliver:

1. A **dead-letter queue** configured with a sensible `maxReceiveCount`, provisioned via IaC (6 marks)
2. A **CloudWatch alarm** on DLQ depth wired to a notification target (5 marks)
3. Consumer changes so a poison message is not retried pointlessly and the failure is logged with enough context to diagnose it (5 marks)
4. `REPLAY.md` — the exact procedure to inspect and replay a DLQ message once the bug is fixed (4 marks)

Evidence: send a bad message, screenshot it landing in the DLQ, and screenshot the alarm in ALARM state.

A3 submission

Push to your branch and open a pull request titled A3 - <your name> containing: `FINDINGS.md`, `REPLAY.md`, an `EVIDENCE/` folder of screenshots, and a `NOTES.md` listing anything you did not finish and how you would have completed it.

Partial credit is real. An honest `NOTES.md` explaining a correct approach you ran out of time to implement earns more than a silent gap. It earns nothing if the work was simply not attempted.

— End of Candidate Paper —